

SQL for SWE Interviews

Intro · Basics · JOINS · Aggregations
Window Functions · CTEs · Classic Problems

Sahil Raj (After Dark)

Contents

1	What Is SQL?	5
1.1	Relational Databases — The Big Picture	5
1.2	Primary Keys and Foreign Keys	6
1.3	SQL Subsets — DDL, DML, DQL, DCL	6
1.4	Data Types — The Essentials	7
1.5	The Schema Used in This Guide	7
1.6	Popular Relational Database Systems	8
2	SQL Fundamentals	9
2.1	SELECT, FROM, WHERE	9
2.2	Filtering — WHERE Clause	9
2.3	Sorting and Limiting	10
2.4	Conditional Logic — CASE WHEN	10
2.5	String & Date Functions	11
3	JOINS	13
3.1	INNER JOIN	13
3.2	LEFT JOIN (Most Common in Interviews)	13
3.3	SELF JOIN	14
3.4	FULL OUTER JOIN	14
3.5	CROSS JOIN & Implicit Cartesian Products	14
3.6	UNION vs UNION ALL	14
4	Aggregations & GROUP BY	17
4.1	COUNT — The Subtleties	17
4.2	GROUP BY	17
4.3	HAVING — Filter After Aggregation	18
4.4	SUM, AVG, MIN, MAX	18
4.5	STRING_AGG / GROUP_CONCAT	18
4.6	WITH ROLLUP & GROUPING SETS	19
5	Window Functions	21
5.1	PARTITION BY and ORDER BY in OVER()	21
5.2	Ranking Functions	21
5.3	LAG and LEAD — Access Adjacent Rows	22
5.4	Running Totals and Moving Averages — Frame Clauses	22
5.5	FIRST_VALUE and LAST_VALUE	22
5.6	Filtering on Window Functions	23
6	Subqueries, CTEs & Classic Problems	25
6.1	Subqueries	25
6.2	CTEs — WITH Clause	26
6.3	Classic Interview Problems	26

6.3.1	Nth Highest Salary	26
6.3.2	Duplicate Emails / Duplicate Detection	27
6.3.3	Consecutive Logins / Streak Detection	27
6.3.4	Department vs Company Average	27
6.3.5	Running Balance / Transaction Ledger	27

Chapter 1

What Is SQL?

SQL Pattern

SQL (Structured Query Language) is the language used to communicate with **relational databases**. You use SQL to store, retrieve, update, and delete data. It is declarative — you describe *what* you want, not *how* to get it. The database engine figures out the how.

SQL is not a programming language in the traditional sense: it has no loops, no if-statements in the procedural sense, and no variables (in standard SQL). It operates on **tables** of data using set-based operations.

1.1 Relational Databases — The Big Picture

A **relational database** organizes data into **tables** (also called *relations*). Think of a table like a spreadsheet with named columns and typed data.

id	name	department	salary	hire_date
1	Sahil Raj	Engineering	90000	2022-07-01
2	Priya Sharma	Design	75000	2021-03-15
3	Aryan Mehta	Engineering	95000	2020-01-10
4	Neha Verma	Product	80000	2023-06-01

Key vocabulary:

Performance Note

Term	Meaning
Table	A named collection of data with a fixed set of columns. Also called a <i>relation</i> .
Row	A single record in a table. Also called a <i>tuple</i> .
Column	A named field with a data type (INTEGER, VARCHAR, DATE, ...). Also called an <i>attribute</i> .
Schema	The structure definition of a table (column names + types). The blueprint, not the data.
Primary Key	A column (or set of columns) that uniquely identifies each row. Never NULL, never duplicate.
Foreign Key	A column that references the primary key of another table. Enforces referential integrity.
NULL	The absence of a value. Not zero, not empty string. A special unknown state.
Index	A data structure (usually B-tree) that speeds up lookups on a column at the cost of write overhead.

1.2 Primary Keys and Foreign Keys

```

1  -- PRIMARY KEY: uniquely identifies each row, cannot be NULL
2  CREATE TABLE departments (
3      id          INTEGER PRIMARY KEY,
4      name        VARCHAR(100) NOT NULL,
5      location    VARCHAR(100)
6  );
7
8  -- FOREIGN KEY: dept_id must match a value in departments.id
9  -- This enforces referential integrity (no orphan employees)
10 CREATE TABLE employees (
11     id          INTEGER PRIMARY KEY,
12     name        VARCHAR(100) NOT NULL,
13     dept_id     INTEGER REFERENCES departments(id),    -- foreign key
14     salary      NUMERIC(10, 2),
15     hire_date   DATE,
16     manager_id  INTEGER REFERENCES employees(id)       -- self-referential FK
17 );

```

Interview Tip

A **foreign key** relationship says: “every value in this column must exist in that other table.” If you try to insert an employee with `dept_id = 99` and no department has `id = 99`, the database rejects it. This is **referential integrity** — a core feature of relational databases that prevents dangling references.

1.3 SQL Subsets — DDL, DML, DQL, DCL

SQL is split into four sublanguages:

Sublanguage	Full Name	Key Statements
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	INSERT, UPDATE, DELETE
DQL	Data Query Language	SELECT
DCL	Data Control Language	GRANT, REVOKE

In interviews, you’ll almost exclusively write DQL (SELECT queries). DDL/DML are good to know but rarely the focus.

1.4 Data Types — The Essentials

```
1  -- Numeric
2  INTEGER / INT           -- whole numbers: -2B to 2B
3  BIGINT                  -- larger whole numbers
4  NUMERIC(p, s)           -- exact decimal with p total digits, s after decimal point
5  FLOAT / REAL            -- approximate floating point (avoid for money!)
6
7  -- Text
8  VARCHAR(n)              -- variable-length string, max n characters
9  CHAR(n)                 -- fixed-length string, padded with spaces
10 TEXT                    -- unlimited length string (PostgreSQL/MySQL)
11
12 -- Date / Time
13 DATE                     -- just the date: 2024-06-25
14 TIME                     -- just the time: 14:30:00
15 TIMESTAMP                -- date + time: 2024-06-25 14:30:00
16 TIMESTAMPTZ              -- timestamp with timezone
17
18 -- Boolean
19 BOOLEAN / BOOL           -- TRUE / FALSE / NULL
20
21 -- Other
22 SERIAL / AUTO_INCREMENT -- auto-incrementing integer (common for primary keys)
23 UUID                     -- universally unique identifier
24 JSON / JSONB             -- structured JSON data (PostgreSQL)
```

1.5 The Schema Used in This Guide

All examples in this book use the following three tables. Mentally load this schema now — you’ll see it on every page.

```
1  -- departments: each row is a department
2  -- id | dept_name | location
3  -- ---+-----+-----
4  -- 1 | Engineering | Remote
5  -- 2 | Design      | NYC
6  -- 3 | Product     | SF
7
8  CREATE TABLE departments (
9      id SERIAL PRIMARY KEY,
10     dept_name VARCHAR(100) NOT NULL,
11     location VARCHAR(100)
12 );
13
14 -- employees: each row is an employee
15 -- id | name | dept_id | salary | hire_date | manager_id
16 -- ---+-----+-----+-----+-----+-----
17 -- 1 | Sahil Raj | 1 | 90000 | 2022-07-01 | NULL
18 -- 2 | Priya Sharma | 2 | 75000 | 2021-03-15 | NULL
19 -- 3 | Aryan Mehta | 1 | 95000 | 2020-01-10 | 1
```

```
-- 4 | Neha Verma | 3 | 80000 | 2023-06-01 | NULL
CREATE TABLE employees (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  dept_id INTEGER REFERENCES departments(id),
  salary NUMERIC(10, 2),
  hire_date DATE,
  manager_id INTEGER REFERENCES employees(id), -- NULL = top-level
  bonus NUMERIC(10, 2) -- can be NULL
);
-- projects: employees are assigned to projects (many-to-many)
CREATE TABLE projects (
  id SERIAL PRIMARY KEY,
  project_name VARCHAR(100) NOT NULL
);
CREATE TABLE project_assignments (
  employee_id INTEGER REFERENCES employees(id),
  project_id INTEGER REFERENCES projects(id),
  PRIMARY KEY (employee_id, project_id) -- composite PK
);
```

1.6 Popular Relational Database Systems

Performance Note

Database	Common Use	SQL Dialect Notes
PostgreSQL	General purpose, default for new projects	Most standard-compliant
MySQL	Web apps, older stacks	GROUP_CONCAT vs STRING_AGG
SQLite	Embedded, local dev, testing	No FULL OUTER JOIN natively
SQL Server	Enterprise, Microsoft stack	TOP n instead of LIMIT n
Oracle	Large enterprise	ROWNUM instead of LIMIT
BigQuery	Analytics at scale (Google Cloud)	Standard SQL + extensions

In interviews: state your dialect assumption upfront. “I’ll write PostgreSQL syntax — let me know if you need something different.” The logic is what matters, not the exact function name.

Common Mistake

SQL is **not** case-sensitive for keywords (SELECT = select = Select), but it **is** case-sensitive for string comparisons in most databases (“Engineering” ≠ “engineering”). Identifiers (table names, column names) may or may not be case-sensitive depending on the OS and database settings.

Chapter 2

SQL Fundamentals

SQL Pattern

SQL queries follow a logical evaluation order that is **different** from write order. Understanding this prevents most beginner mistakes.

Write order: SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT

Eval order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Consequence: you cannot use a SELECT alias in a WHERE clause (WHERE runs before SELECT). You CAN use it in ORDER BY.

2.1 SELECT, FROM, WHERE

```
1  -- Basic retrieval
2  SELECT name, salary, department
3  FROM employees
4  WHERE salary > 50000
5     AND department = 'Engineering';
6
7  -- DISTINCT: remove duplicate rows in the result
8  SELECT DISTINCT department
9  FROM employees;
10
11 -- Column aliases (AS is optional)
12 SELECT name, salary * 12 AS annual_salary
13 FROM employees;
14
15 -- Table aliases (required when joining same table to itself)
16 SELECT e.name, e.salary
17 FROM employees AS e
18 WHERE e.salary > 70000;
```

2.2 Filtering — WHERE Clause

```
1  -- Comparison operators: =, <>, !=, <, >, <=, >=
2  SELECT * FROM employees WHERE salary <> 0;
3
4  -- Range: BETWEEN is inclusive on both ends
5  SELECT * FROM employees WHERE salary BETWEEN 50000 AND 100000;
6
7  -- Set membership
8  SELECT * FROM employees WHERE department IN ('Engineering', 'Design', 'Product');
9
10 -- Pattern matching: % = any sequence, _ = exactly one char
11 SELECT * FROM employees WHERE name LIKE 'Sa%';      -- starts with 'Sa'
```

```

12 SELECT * FROM employees WHERE name LIKE '_a%';           -- second char is 'a'
13 SELECT * FROM employees WHERE name NOT LIKE '%test%';
14
15 -- NULL handling: NEVER use = NULL, always IS NULL / IS NOT NULL
16 SELECT * FROM employees WHERE manager_id IS NULL;        -- top-level employees
17 SELECT * FROM employees WHERE bonus IS NOT NULL;
18
19 -- COALESCE: return first non-NULL value
20 SELECT name, COALESCE(bonus, 0) AS bonus
21 FROM employees;

```

Common Mistake

NULL = NULL evaluates to NULL (unknown), not TRUE. This is why WHERE col = NULL always returns 0 rows. Always use IS NULL or IS NOT NULL.

2.3 Sorting and Limiting

```

1  -- ORDER BY: ASC is default, NULL sorts last in ASC (most databases)
2  SELECT name, salary
3  FROM employees
4  ORDER BY salary DESC, name ASC;    -- sort by salary desc, break ties by name
5
6  -- LIMIT / OFFSET for pagination
7  SELECT name, salary
8  FROM employees
9  ORDER BY salary DESC
10 LIMIT 10 OFFSET 20;    -- page 3 of 10-per-page results (0-indexed: skip 20)
11
12 -- LIMIT is MySQL/PostgreSQL syntax
13 -- SQL Server uses: SELECT TOP 10 ...
14 -- Oracle uses:      WHERE ROWNUM <= 10 (or FETCH FIRST 10 ROWS ONLY)

```

2.4 Conditional Logic — CASE WHEN

```

1  -- Simple CASE (like switch)
2  SELECT name,
3         CASE department
4             WHEN 'Engineering' THEN 'Tech'
5             WHEN 'Design'      THEN 'Creative'
6             ELSE                'Other'
7         END AS dept_group
8  FROM employees;
9
10 -- Searched CASE (like if-elseif)
11 SELECT name, salary,
12        CASE
13            WHEN salary >= 100000 THEN 'Senior'
14            WHEN salary >= 60000  THEN 'Mid'
15            ELSE                  'Junior'
16        END AS level
17 FROM employees;
18
19 -- Use CASE inside aggregates (conditional aggregation)
20 SELECT
21     COUNT(CASE WHEN department = 'Engineering' THEN 1 END) AS eng_count,
22     COUNT(CASE WHEN department = 'Design'      THEN 1 END) AS design_count
23 FROM employees;

```

Interview Tip

Conditional aggregation (COUNT(CASE WHEN ... THEN 1 END)) is one of the most commonly tested interview patterns. It lets you pivot row data into columns without a real PIVOT operator.

2.5 String & Date Functions

```
1  -- String functions (PostgreSQL / MySQL syntax)
2  SELECT UPPER(name), LOWER(email), LENGTH(name)
3  FROM employees;
4
5  SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees; -- MySQL
6  SELECT first_name || ' ' || last_name AS full_name FROM employees;      -- PostgreSQL
7
8  SELECT SUBSTRING(email, 1, POSITION('@' IN email) - 1) AS username FROM employees;
9  SELECT TRIM(name), LTRIM(name), RTRIM(name) FROM employees;
10
11 -- Date functions
12 SELECT CURRENT_DATE, CURRENT_TIMESTAMP;
13 SELECT EXTRACT(YEAR FROM hire_date) AS hire_year FROM employees;
14 SELECT hire_date + INTERVAL '30 days' AS trial_end FROM employees;
15 SELECT DATE_PART('day', CURRENT_DATE - hire_date) AS days_employed FROM employees;
```

Performance Note

Date/string functions vary significantly between databases. In interviews, state your assumption (“I’ll use PostgreSQL syntax here”) and proceed. Interviewers care about the logic, not the exact function name.

Chapter 3

JOINS

SQL Pattern

A JOIN combines rows from two tables based on a related column. The join type determines what happens when there is no match.

Join Type	Returns	Unmatched rows
INNER JOIN	Rows matching in both tables	Excluded from both sides
LEFT JOIN	All left rows + matched right rows	Right side gets NULL
RIGHT JOIN	All right rows + matched left rows	Left side gets NULL
FULL OUTER JOIN	All rows from both tables	Unmatched side gets NULL
CROSS JOIN	Cartesian product of both tables	(no join condition)
SELF JOIN	Table joined with itself	Use table aliases

3.1 INNER JOIN

```
1 -- employees: id, name, dept_id, salary
2 -- departments: id, dept_name, manager_id
3
4 -- Only employees who have a matching department
5 SELECT e.name, e.salary, d.dept_name
6 FROM employees e
7 INNER JOIN departments d ON e.dept_id = d.id;
8
9 -- Multiple JOINS: chain them
10 SELECT e.name, d.dept_name, p.project_name
11 FROM employees e
12 JOIN departments d ON e.dept_id = d.id
13 JOIN project_assignments pa ON e.id = pa.employee_id
14 JOIN projects p ON pa.project_id = p.id;
```

3.2 LEFT JOIN (Most Common in Interviews)

```
1 -- All employees, including those with no department (dept_id is NULL)
2 SELECT e.name, COALESCE(d.dept_name, 'No Department') AS dept
3 FROM employees e
4 LEFT JOIN departments d ON e.dept_id = d.id;
5
6 -- Classic: find employees who have NO manager in the managers table
```

```

7 SELECT e.name
8 FROM employees e
9 LEFT JOIN managers m ON e.id = m.employee_id
10 WHERE m.employee_id IS NULL;      -- NULL after LEFT JOIN = no match existed

```

Interview Tip

The “IS NULL after LEFT JOIN” trick is one of the most common interview patterns. It’s equivalent to a set difference: employees who are NOT in the managers table. Much faster than NOT IN on large tables.

3.3 SELF JOIN

```

1 -- Employee-manager hierarchy: employees table has manager_id FK to same table
2 -- Find each employee's manager name
3 SELECT e.name AS employee, m.name AS manager
4 FROM employees e
5 LEFT JOIN employees m ON e.manager_id = m.id;
6
7 -- Find all pairs of employees in the same department (avoid duplicate pairs)
8 SELECT a.name AS emp1, b.name AS emp2, a.department
9 FROM employees a
10 JOIN employees b ON a.department = b.department AND a.id < b.id;

```

3.4 FULL OUTER JOIN

```

1 -- All employees + all departments, even if no match exists on either side
2 SELECT e.name, d.dept_name
3 FROM employees e
4 FULL OUTER JOIN departments d ON e.dept_id = d.id;
5 -- Unmatched employees: dept_name = NULL
6 -- Unmatched departments: name = NULL
7
8 -- Emulate FULL OUTER JOIN in MySQL (which doesn't support it):
9 SELECT e.name, d.dept_name FROM employees e LEFT JOIN departments d ON e.dept_id = d.id
10 UNION
11 SELECT e.name, d.dept_name FROM employees e RIGHT JOIN departments d ON e.dept_id = d.id;

```

3.5 CROSS JOIN & Implicit Cartesian Products

```

1 -- Explicit CROSS JOIN: every row of A paired with every row of B
2 -- 10 rows x 5 rows = 50 output rows
3 SELECT a.color, b.size
4 FROM colors a
5 CROSS JOIN sizes b;
6
7 -- Use case: generate a date spine (all dates in a range) when joined with a calendar table
8 -- Use case: generate all possible combinations (e.g., menu items x toppings)

```

Common Mistake

A missing JOIN condition accidentally creates a cartesian product. If your query runs for way too long and returns millions of rows, check that all your JOINS have ON clauses.

3.6 UNION vs UNION ALL

```
1  -- UNION: combine results of two queries, removing duplicates (implicit DISTINCT)
2  -- UNION ALL: combine results, keeping all rows including duplicates (faster)
3  SELECT name FROM employees_us
4  UNION ALL
5  SELECT name FROM employees_eu;    -- keeps duplicates, no dedup cost
6
7  -- Rules: both queries must have the same number of columns, same data types
8  -- Column names come from the FIRST query
9
10 -- INTERSECT: rows that appear in BOTH queries
11 -- EXCEPT (or MINUS in Oracle): rows in first query but not second
12 SELECT id FROM 2023_users
13 INTERSECT
14 SELECT id FROM 2024_users;    -- users active in both years
```

Performance Note

UNION vs JOIN: JOIN combines columns (horizontal); UNION combines rows (vertical). A common interview confusion.

Performance: UNION ALL is significantly faster than UNION for large datasets because it skips the deduplication sort. Use UNION only when duplicates are actually a problem.

Chapter 4

Aggregations & GROUP BY

SQL Pattern

Aggregation functions collapse multiple rows into a single value. They're used with GROUP BY to produce one row per group. Key rule: any column in SELECT that is **not** inside an aggregate function **must** appear in GROUP BY.

Aggregate functions: COUNT, SUM, AVG, MIN, MAX, STRING_AGG, ARRAY_AGG.

4.1 COUNT — The Subtleties

```
1  -- COUNT(*): count all rows, including NULLs
2  SELECT COUNT(*) FROM employees;           -- total rows
3
4  -- COUNT(column): count non-NULL values in that column
5  SELECT COUNT(bonus) FROM employees;       -- employees who have a bonus
6
7  -- COUNT(DISTINCT column): count unique non-NULL values
8  SELECT COUNT(DISTINCT department) FROM employees; -- how many departments exist
9
10 -- Difference matters when column has NULLs!
11 -- employees: [100, 200, NULL, 300, NULL]
12 -- COUNT(*)           = 5
13 -- COUNT(bonus)       = 3
14 -- COUNT(DISTINCT bonus) = 3 (NULL is excluded from DISTINCT count too)
```

Common Mistake

COUNT(DISTINCT *) is invalid syntax. To count distinct rows, use a subquery:
SELECT COUNT(*) FROM (SELECT DISTINCT col1, col2 FROM t) sub;

4.2 GROUP BY

```
1  -- Average salary by department
2  SELECT department, AVG(salary) AS avg_salary, COUNT(*) AS headcount
3  FROM employees
4  GROUP BY department;
5
6  -- Group by multiple columns
7  SELECT department, job_title, AVG(salary) AS avg_salary
8  FROM employees
9  GROUP BY department, job_title;
10
11 -- Group by expression (not just a column)
```

```

12 SELECT EXTRACT(YEAR FROM hire_date) AS hire_year, COUNT(*) AS new_hires
13 FROM employees
14 GROUP BY EXTRACT(YEAR FROM hire_date)
15 ORDER BY hire_year;
16
17 -- Can reference SELECT alias in ORDER BY (ORDER BY runs after SELECT)
18 SELECT department, SUM(salary) AS total_pay
19 FROM employees
20 GROUP BY department
21 ORDER BY total_pay DESC;      -- 'total_pay' alias is valid here

```

4.3 HAVING — Filter After Aggregation

```

1  -- WHERE filters rows BEFORE grouping; HAVING filters groups AFTER aggregation
2  -- Find departments with average salary > 80000
3  SELECT department, AVG(salary) AS avg_salary
4  FROM employees
5  WHERE status = 'active'          -- filter rows first (before GROUP BY)
6  GROUP BY department
7  HAVING AVG(salary) > 80000;      -- filter groups (after GROUP BY)
8
9  -- Departments with more than 5 employees, ordered by headcount
10 SELECT department, COUNT(*) AS cnt
11 FROM employees
12 GROUP BY department
13 HAVING COUNT(*) > 5
14 ORDER BY cnt DESC;

```

Interview Tip

Rule of thumb: If you're filtering on an aggregate (COUNT, SUM, AVG, etc.), use HAVING. If you're filtering on a raw column value, use WHERE. You can have both in the same query.

4.4 SUM, AVG, MIN, MAX

```

1  SELECT
2      SUM(salary)           AS total_payroll,
3      AVG(salary)          AS avg_salary,
4      MIN(salary)          AS min_salary,
5      MAX(salary)          AS max_salary,
6      MAX(salary) - MIN(salary) AS salary_range
7  FROM employees;
8
9  -- Conditional aggregation: pivot rows into columns without PIVOT keyword
10 SELECT
11     SUM(CASE WHEN year = 2023 THEN revenue ELSE 0 END) AS rev_2023,
12     SUM(CASE WHEN year = 2024 THEN revenue ELSE 0 END) AS rev_2024,
13     SUM(CASE WHEN year = 2024 THEN revenue ELSE 0 END) -
14     SUM(CASE WHEN year = 2023 THEN revenue ELSE 0 END) AS yoy_growth
15 FROM sales;

```

4.5 STRING_AGG / GROUP_CONCAT

```

1  -- Concatenate all names in a group into a comma-separated string
2  -- PostgreSQL / SQL Server:
3  SELECT department, STRING_AGG(name, ', ' ORDER BY name) AS members
4  FROM employees
5  GROUP BY department;
6
7  -- MySQL:

```

```
8 SELECT department, GROUP_CONCAT(name ORDER BY name SEPARATOR ', ') AS members
9 FROM employees
10 GROUP BY department;
```

4.6 WITH ROLLUP & GROUPING SETS

```
1 -- ROLLUP: subtotals + grand total automatically
2 SELECT department, job_title, SUM(salary)
3 FROM employees
4 GROUP BY ROLLUP(department, job_title);
5 -- Produces: per-(dept, title), per-dept subtotals, and grand total
6 -- NULL in result = "all" for that level
7
8 -- GROUPING SETS: explicit list of which combinations to aggregate
9 SELECT department, job_title, SUM(salary)
10 FROM employees
11 GROUP BY GROUPING SETS ((department), (job_title), ());
12 -- () = grand total
```

Performance Note

ROLLUP and GROUPING SETS are rarely tested in intern interviews but may appear in analytics-focused roles. Know that they exist and can replace multiple UNION ALL queries.

Chapter 5

Window Functions

SQL Pattern

Window functions perform calculations **across a set of rows related to the current row**, without collapsing them into a single output row (unlike GROUP BY). The window is defined by the OVER() clause.

Syntax: function_name(...) OVER ([PARTITION BY ...] [ORDER BY ...] [frame])

Key insight: Window functions run *after* WHERE, GROUP BY, and HAVING, but before the final SELECT. You can use them in SELECT and ORDER BY but **not** in WHERE or HAVING.

5.1 PARTITION BY and ORDER BY in OVER()

```
1  -- employees: id, name, department, salary, hire_date
2
3  -- Rank salary within each department
4  SELECT name, department, salary,
5         RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank
6  FROM employees;
7  -- PARTITION BY department: restart ranking for each department
8  -- ORDER BY salary DESC: rank by salary, highest = rank 1
9
10 -- Without PARTITION BY: rank across entire table
11 SELECT name, salary,
12        RANK() OVER (ORDER BY salary DESC) AS overall_rank
13 FROM employees;
```

5.2 Ranking Functions

```
1  -- ROW_NUMBER: unique sequential number (no ties), arbitrary tiebreak
2  -- RANK: tied rows get same rank; next rank skips numbers (1,1,3,4)
3  -- DENSE_RANK: tied rows get same rank; next rank is consecutive (1,1,2,3)
4
5  SELECT name, salary,
6         ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
7         RANK() OVER (ORDER BY salary DESC) AS rnk,
8         DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk
9  FROM employees;
10
11 -- Typical result for salaries [100k, 100k, 90k, 80k]:
12 -- ROW_NUMBER:  1, 2, 3, 4
13 -- RANK:        1, 1, 3, 4
14 -- DENSE_RANK:  1, 1, 2, 3
15
16 -- NTILE(n): divide rows into n buckets (quartiles, deciles, etc.)
```

```

17 SELECT name, salary,
18        NTILE(4) OVER (ORDER BY salary DESC) AS quartile
19 FROM employees;
20 -- Quartile 1 = top 25%, Quartile 4 = bottom 25%

```

Interview Tip

Most common interview pattern: Find the Nth highest salary per department. Use `DENSE_RANK()` inside a subquery, then filter: `WHERE dense_rnk = N`. Never use `RANK()` for this — ties could skip the Nth value.

5.3 LAG and LEAD — Access Adjacent Rows

```

1 -- LAG(col, offset, default): value from N rows BEFORE current row
2 -- LEAD(col, offset, default): value from N rows AFTER current row
3
4 -- Month-over-month revenue change
5 SELECT month, revenue,
6        LAG(revenue, 1, 0) OVER (ORDER BY month) AS prev_month_rev,
7        revenue - LAG(revenue, 1, 0) OVER (ORDER BY month) AS mom_change
8 FROM monthly_revenue;
9
10 -- Compare each employee's salary with their direct predecessor (by hire date)
11 SELECT name, hire_date, salary,
12        LAG(salary) OVER (PARTITION BY department ORDER BY hire_date) AS prev_hire_salary,
13        salary - LAG(salary) OVER (PARTITION BY department ORDER BY hire_date) AS salary_diff
14 FROM employees;

```

5.4 Running Totals and Moving Averages — Frame Clauses

```

1 -- Running total of salary (cumulative sum from start to current row)
2 SELECT name, hire_date, salary,
3        SUM(salary) OVER (ORDER BY hire_date
4                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
5 FROM employees;
6
7 -- 3-month moving average
8 SELECT month, revenue,
9        AVG(revenue) OVER (ORDER BY month
10                          ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moving_avg_3
11 FROM monthly_revenue;
12
13 -- Frame clause syntax:
14 -- ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW (default for SUM/AVG with ORDER BY)
15 -- ROWS BETWEEN 2 PRECEDING AND CURRENT ROW (rolling window)
16 -- ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING (entire partition)
17 -- RANGE vs ROWS: RANGE groups identical ORDER BY values together; ROWS treats each row
   independently

```

5.5 FIRST_VALUE and LAST_VALUE

```

1 -- First/last salary hired in each department
2 SELECT name, department, salary,
3        FIRST_VALUE(name) OVER (PARTITION BY department ORDER BY hire_date) AS first_hire,
4        LAST_VALUE(name) OVER (PARTITION BY department ORDER BY hire_date
5                              ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
6        last_hire
7 FROM employees;
8 -- LAST_VALUE needs explicit UNBOUNDED FOLLOWING frame, otherwise it defaults
9 -- to CURRENT ROW and gives the current row's value (a common gotcha!)

```

Common Mistake

`LAST_VALUE` uses the default frame `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, so without the explicit `UNBOUNDED FOLLOWING` frame, it returns the value of the current row, not the actual last row of the partition. Always specify the frame for `LAST_VALUE`.

5.6 Filtering on Window Functions

```
1  -- Cannot use window function in WHERE or HAVING. Wrap in a subquery or CTE.
2
3  -- Find employees who earn more than their department average
4  SELECT name, department, salary
5  FROM (
6      SELECT name, department, salary,
7             AVG(salary) OVER (PARTITION BY department) AS dept_avg
8      FROM employees
9  ) sub
10 WHERE salary > dept_avg;
11
12 -- Find the top-2 earners per department using DENSE_RANK
13 SELECT name, department, salary
14 FROM (
15     SELECT name, department, salary,
16            DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dr
17     FROM employees
18 ) sub
19 WHERE dr <= 2;
```


Chapter 6

Subqueries, CTEs & Classic Problems

SQL Pattern

Subqueries and CTEs are the primary tools for structuring complex SQL. A **CTE** (WITH clause) is almost always preferable to a deeply nested subquery: same semantics, far more readable. Use subqueries for simple scalar lookups; use CTEs when the logic has more than one step.

6.1 Subqueries

```
1  -- Scalar subquery: returns exactly one row, one column
2  SELECT name, salary,
3         (SELECT AVG(salary) FROM employees) AS company_avg
4  FROM employees;
5
6  -- Subquery in WHERE
7  SELECT name FROM employees
8  WHERE salary > (SELECT AVG(salary) FROM employees);
9
10 -- IN subquery: test set membership
11 SELECT name FROM employees
12 WHERE dept_id IN (SELECT id FROM departments WHERE location = 'Remote');
13
14 -- EXISTS subquery: tests whether a subquery returns any rows (usually faster than IN for
    large sets)
15 SELECT d.dept_name
16 FROM departments d
17 WHERE EXISTS (
18     SELECT 1 FROM employees e WHERE e.dept_id = d.id AND e.salary > 100000
19 );
20
21 -- Correlated subquery: references the outer query's row (runs once per outer row -- can be
    slow!)
22 SELECT e.name, e.salary
23 FROM employees e
24 WHERE e.salary > (
25     SELECT AVG(salary) FROM employees WHERE department = e.department -- correlated!
26 );
```

Performance Note**EXISTS vs IN:**

- IN materializes the subquery result first, then checks membership. Slow if subquery is large.
- EXISTS short-circuits as soon as it finds one matching row. Faster for large right-hand sets.
- NOT IN can return unexpected results if the subquery contains NULLs (x NOT IN (1, NULL) is always NULL/unknown). Prefer NOT EXISTS.

6.2 CTEs — WITH Clause

```

1  -- Basic CTE: like a named subquery, readable and reusable within the query
2  WITH dept_stats AS (
3      SELECT department, AVG(salary) AS avg_sal, COUNT(*) AS cnt
4      FROM employees
5      GROUP BY department
6  )
7  SELECT e.name, e.salary, ds.avg_sal,
8         e.salary - ds.avg_sal AS salary_vs_avg
9  FROM employees e
10 JOIN dept_stats ds ON e.department = ds.department
11 WHERE e.salary > ds.avg_sal;
12
13 -- Multiple CTEs (chain them)
14 WITH
15 high_earners AS (
16     SELECT id, name, salary FROM employees WHERE salary > 100000
17 ),
18 their_departments AS (
19     SELECT e.name, d.dept_name
20     FROM high_earners e
21     JOIN departments d ON e.dept_id = d.id
22 )
23 SELECT * FROM their_departments;

```

```

1  -- Recursive CTE: traverse hierarchies (org chart, categories, graphs)
2  WITH RECURSIVE org AS (
3      -- Base case: top-level employees (no manager)
4      SELECT id, name, manager_id, 1 AS depth
5      FROM employees WHERE manager_id IS NULL
6
7      UNION ALL
8
9      -- Recursive case: join with their direct reports
10     SELECT e.id, e.name, e.manager_id, o.depth + 1
11     FROM employees e
12     JOIN org o ON e.manager_id = o.id
13 )
14 SELECT name, depth FROM org ORDER BY depth, name;

```

6.3 Classic Interview Problems

6.3.1 Nth Highest Salary

```

1  -- Method 1: DENSE_RANK (handles ties correctly)
2  SELECT salary AS NthHighest
3  FROM (
4      SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS dr
5      FROM employees
6  ) sub
7  WHERE dr = 3;    -- 3rd highest
8

```

```

9  -- Method 2: OFFSET-LIMIT (simpler, but fails if N=1 with ties)
10 SELECT DISTINCT salary FROM employees
11 ORDER BY salary DESC
12 LIMIT 1 OFFSET 2;      -- skip 2, take 1 (0-indexed)

```

6.3.2 Duplicate Emails / Duplicate Detection

```

1  -- Find emails that appear more than once
2  SELECT email, COUNT(*) AS cnt
3  FROM users
4  GROUP BY email
5  HAVING COUNT(*) > 1;
6
7  -- Delete duplicates, keep the row with the smallest id
8  DELETE FROM users
9  WHERE id NOT IN (
10     SELECT MIN(id) FROM users GROUP BY email
11 );

```

6.3.3 Consecutive Logins / Streak Detection

```

1  -- Classic island-and-gaps problem: find streaks of consecutive login days
2  -- Step 1: assign row number per user ordered by date
3  -- Step 2: date - row_number is constant within a streak (the "island")
4  WITH numbered AS (
5      SELECT user_id, login_date,
6             ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) AS rn
7      FROM logins
8  ),
9  islands AS (
10     SELECT user_id, login_date,
11            login_date - CAST(rn AS INT) AS island_key    -- constant within streak
12     FROM numbered
13  )
14 SELECT user_id, MIN(login_date) AS streak_start,
15        MAX(login_date) AS streak_end,
16        COUNT(*) AS streak_length
17 FROM islands
18 GROUP BY user_id, island_key
19 HAVING COUNT(*) >= 3;    -- only streaks of 3+ days

```

6.3.4 Department vs Company Average

```

1  WITH averages AS (
2      SELECT department,
3             AVG(salary) AS dept_avg,
4             AVG(AVG(salary)) OVER () AS company_avg    -- avg of dept avgs
5      FROM employees
6      GROUP BY department
7  )
8  SELECT department,
9         ROUND(dept_avg, 2) AS dept_avg,
10        ROUND(company_avg, 2) AS company_avg,
11        CASE WHEN dept_avg > company_avg THEN 'Above' ELSE 'Below' END AS vs_company
12 FROM averages
13 ORDER BY dept_avg DESC;

```

6.3.5 Running Balance / Transaction Ledger

```

1  -- Compute running account balance after each transaction
2  SELECT transaction_id, account_id, amount, transaction_date,
3         SUM(amount) OVER (

```

```
4     PARTITION BY account_id
5     ORDER BY transaction_date, transaction_id    -- break ties by id
6     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
7   ) AS running_balance
8 FROM transactions;
```

Interview Tip

Interview strategy: Before writing SQL, say aloud: “Let me break this into steps: first I need [X], then join with [Y], then filter for [Z].” This maps directly to CTEs and shows structured thinking. Interviewers evaluate your thought process as much as your final query.